

Understand Runtime Polymorphism Through Parent Reference and Child Object (Interview Notes)

What is Runtime Polymorphism?

Runtime Polymorphism (also called **Dynamic Method Dispatch** or **Method Overriding**) occurs when a **parent class reference** refers to a **child class object**, and the **overridden method is selected at runtime**.

The JVM decides **which method to execute based on the actual object**, not the reference type.

Basic Example

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {

        Animal a = new Dog();

        a.sound();
    }
}
```

Output

```
Dog barks
```

Although the reference is `Animal`, the object is `Dog`, so the `Dog` version of `sound()` executes.

Why Does This Happen?

```
Animal a = new Dog();
```

Here,

- **Reference Type** → `Animal`
- **Object Type** → `Dog`

During compilation:

- Compiler checks whether `Animal` has the `sound()` method.

During runtime:

- JVM checks the **actual object (`Dog`)**.
 - Calls `Dog`'s overridden `sound()` method.
-

Real-world Example

Imagine a remote control.

```
Remote (Reference)
```

```
↓
```

```
TV
```

```
AC
```

```
Projector
```

The **same remote (parent reference)** can control **different devices (child objects)**.

The action performed depends on **which device is actually connected**, just like the JVM chooses the method based on the object's actual type.

Important Interview Points

- Runtime polymorphism is achieved through **method overriding**, not method overloading.
 - A **parent class reference can point to a child class object**.
 - The **actual object's method** is executed, not the reference type's method.
 - Method selection happens **at runtime**, which is why it is called runtime polymorphism.
 - Only **overridden instance methods** participate in runtime polymorphism.
 - **Static, private, and final methods cannot be overridden**, so they do not support runtime polymorphism.
 - Runtime polymorphism enables **loose coupling, extensibility, and flexible code design**, making it a core principle of object-oriented programming.
-

Tricky Interview Questions

Q1. Which determines the method execution: reference type or object type?

Answer:

Object type determines which overridden method executes.

Q2. What determines which members can be accessed?

Answer:

The **reference type** determines what methods and fields are accessible at compile time.

Example:

```
Animal a = new Dog();
```

You can only call methods declared in Animal (unless you cast), but overridden methods execute from Dog.

Q3. Does runtime polymorphism work with variables (fields)?

Answer: No.

Fields are resolved using the **reference type**, not the object type.

```
class Animal {  
    String name = "Animal";  
}  
  
class Dog extends Animal {  
    String name = "Dog";  
}  
  
Animal a = new Dog();  
System.out.println(a.name);
```

Output

```
Animal
```

Fields are **hidden**, not overridden.

Q4. Can constructors be overridden?

Answer: No.

Constructors are **not inherited**, so runtime polymorphism does not apply.

Q5. Do static methods support runtime polymorphism?

Answer: No.

Static methods are **hidden**, not overridden, and are resolved at **compile time**.

Q6. What if the child class doesn't override the method?

Answer:

The parent class method is executed.

Q7. Can we create a parent object and assign it to a child reference?

```
Dog d = new Animal();
```

Answer: No.

This causes a **compile-time error** because a parent object cannot be assigned to a child reference.

Q8. What is Dynamic Method Dispatch?

Answer:

It is the JVM's process of selecting the **overridden method at runtime** based on the **actual object**.

Interview Summary

- **Runtime polymorphism is achieved through method overriding.**
- A **parent reference** can point to a **child object**.
- The **JVM selects the overridden method at runtime** based on the actual object's type.
- The **reference type controls method accessibility**, while the **object type controls method execution**.
- Fields, constructors, static methods, private methods, and final methods do **not** participate in runtime polymorphism.
- Runtime polymorphism is one of the key features that enables **flexible and extensible object-oriented designs**.

Interview One-Liner:

"In runtime polymorphism, a parent reference can refer to a child object, and the JVM invokes the overridden method of the actual object at runtime through dynamic method dispatch."